🐍 Comprehensive Python Developer Syllabus

Phase 1: Foundation Reinforcement (Week 1-2)

1.1 Core Fundamentals Review

- Variables, Data Types, and Type Conversion
- Control Flow (if/else, loops)
- Functions (parameters, return values, scope)
- **Global vs Local Variables** (you mentioned this - solidify it!)
- Data Structures: Lists, Tuples, Sets, Dictionaries

YouTube Resources:

- **Corey Schafer - Python Tutorial Series**: Search for "Corey Schafer Python Beginner" (20-30 min videos, excellent explanations)
- **Programming with Mosh - Python Tutorial for Beginners**: Full course available

1.2 Object-Oriented Programming (OOP) - Deep Dive

- Classes and Objects
- Attributes and Methods
- The `__init__` constructor
- Inheritance (single, multiple, multi-level)
- Polymorphism and Method Overriding
- Encapsulation (public, private, protected)
- Abstract Classes and Interfaces
- **Special/Magic/Dunder Methods** (`__str__`, `__repr__`, `__len__`, `__add__`, etc.)

YouTube Resources:

- **Corey Schafer - OOP Series**: Search "Corey Schafer Python OOP"
- **Tech With Tim - Python OOP Tutorial**: Comprehensive 6-part series

---

Phase 2: Intermediate Python (Week 3-5)

2.1 Generators ⭐

- What are generators and why use them?
- `yield` keyword
- Generator expressions vs List comprehensions
- Memory efficiency with generators
- `itertools` module

YouTube Resources:

- **Corey Schafer - Python Generators**: Search "Corey Schafer Generators"
- **Tech With Tim - Generators Explained**

2.2 Decorators ⭐

- Functions as first-class objects
- Function closures
- Basic decorator syntax (`@decorator`)
- Decorators with arguments
- Class-based decorators
- `functools.wraps`
- Common use cases (timing, logging, authentication)

YouTube Resources:

- **Corey Schafer - Python Decorators Tutorial**: Essential watching
- **Real Python - Primer on Python Decorators**
- **Tech With Tim - Python Decorators**

2.3 Context Managers ⭐

- The `with` statement
- `__enter__` and `__exit__` methods
- Creating custom context managers
- `contextlib` module

- `@contextmanager` decorator
- Exception handling in context managers

YouTube Resources:

- **Corey Schafer - Context Managers**: Search "Corey Schafer Context Managers"
- **Real Python - Context Managers**

2.4 Iterators and Iterables

- Understanding iteration protocol
- `__iter__` and `__next__` methods
- Creating custom iterators
- Difference between iterators and iterables

YouTube Resources:

- **Corey Schafer - Iterators and Iterables**

2.5 Lambda Functions and Functional Programming

- Lambda expressions
- `map()`, `filter()`, `reduce()`
- List/Dict/Set comprehensions (advanced)
- Nested comprehensions

YouTube Resources:

- **Corey Schafer - Lambda, Map, Filter, Reduce**

Phase 3: Advanced Python Concepts (Week 6-8)

3.1 Advanced OOP Concepts

- Metaclasses
- Property decorators (@property, @setter, @deleter)
- Class methods vs Static methods
- Method Resolution Order (MRO)
- Data Classes (`@dataclass`)

- Slots (`__slots__`)

YouTube Resources:

- **ArjanCodes - Advanced Python Playlist**
- **mCoding - Python Advanced Topics**

3.2 Error and Exception Handling

- Try, Except, Else, Finally
- Creating custom exceptions
- Exception chaining
- Context managers for resource management
- Best practices for exception handling

YouTube Resources:

- **Corey Schafer - Python Exceptions**

3.3 File Handling and I/O

- Reading/Writing files (text and binary)
- Working with CSV, JSON, XML
- File paths with `pathlib`
- Context managers for files

YouTube Resources:

- **Corey Schafer - File Objects**
- **Corey Schafer - Working with JSON Data**

3.4 Regular Expressions (Regex)

- Pattern matching basics
- `re` module functions
- Common patterns
- Greedy vs non-greedy matching

YouTube Resources:

- **Corey Schafer - Regular Expressions**

3.5 Working with Modules and Packages

- Creating modules
- `__init__.py` and package structure
- Absolute vs relative imports
- `if __name__ == "__main__"`
- Virtual environments

YouTube Resources:

- **Corey Schafer - Python Modules and Packages**
- **Corey Schafer - Virtual Environments**

Phase 4: Asynchronous Programming (Week 9-10)

4.1 Threading and Multiprocessing

- Global Interpreter Lock (GIL)
- `threading` module
- `multiprocessing` module
- Thread-safe operations
- When to use threads vs processes

YouTube Resources:

- **Corey Schafer - Python Threading**
- **Corey Schafer - Python Multiprocessing**

4.2 Asyncio and Async/Await

- Coroutines
- `async` and `await` keywords
- Event loops
- `asyncio` module
- Async context managers
- Async generators

YouTube Resources:

- **ArjanCodes - AsyncIO Complete Guide**
- **mCoding - Asyncio is Simple**

Phase 5: Testing and Debugging (Week 11)

5.1 Unit Testing

- `unittest` framework
- `pytest` framework (industry standard)
- Test fixtures
- Mocking and patching
- Test coverage
- Test-Driven Development (TDD)

YouTube Resources:

- **Corey Schafer - Python Unit Testing**
- **ArjanCodes - Python Testing**

5.2 Debugging Techniques

- Using `pdb` debugger
- IDE debugging tools
- Logging best practices
- Profiling code performance

YouTube Resources:

- **Corey Schafer - Python Logging**

Phase 6: Python for Development (Week 12-14)

6.1 Working with APIs

- HTTP requests with `requests` library
- RESTful API concepts

- JSON parsing
- Authentication (API keys, OAuth)
- Rate limiting

YouTube Resources:

- **Corey Schafer - Python Requests Tutorial**
- **Tech With Tim - APIs in Python**

6.2 Database Interaction

- SQL basics review
- `sqlite3` module
- SQLAlchemy ORM
- Database migrations
- Connection pooling

YouTube Resources:

- **Corey Schafer - Python SQLite Tutorial**
- **Tech With Tim - SQLAlchemy**

6.3 Web Development Basics

- Flask framework basics
- Django framework basics
- Templates and routing
- Forms and validation
- Session management

YouTube Resources:

- **Corey Schafer - Flask Tutorial Series**
- **Corey Schafer - Django Tutorial Series**
- **Tech With Tim - Flask Tutorial**

Phase 7: Professional Development Practices (Week 15-16)

7.1 Code Quality and Style

- PEP 8 style guide
- Type hints and annotations
- `mypy` for type checking
- Code linters (pylint, flake8)
- Code formatters (black, autopep8)

YouTube Resources:

- **ArjanCodes - Code Smell Series**
- **mCoding - Type Hints**

7.2 Design Patterns

- Singleton pattern
- Factory pattern
- Observer pattern
- Strategy pattern
- Dependency injection

YouTube Resources:

- **ArjanCodes - Design Patterns Playlist**

7.3 Version Control

- Git fundamentals
- Branching strategies
- Pull requests and code reviews
- `.gitignore` best practices

YouTube Resources:

- **Corey Schafer - Git Tutorial**
- **Tech With Tim - Git and GitHub**

7.4 Package Management and Distribution

- `pip` and `requirements.txt`

- `setup.py` and `pyproject.toml`
- Creating and publishing packages to PyPI
- Poetry for dependency management

YouTube Resources:

- **ArjanCodes - Python Packages**

Phase 8: Data Handling (Week 17-18)

8.1 Data Manipulation

- NumPy basics
- Pandas basics
- Data cleaning and preprocessing
- Working with large datasets

YouTube Resources:

- **Keith Galli - Complete Python Pandas Data Science Tutorial**
- **freeCodeCamp - NumPy Tutorial**

8.2 Data Serialization

- JSON, XML, YAML
- Pickle and joblib
- Protocol Buffers basics

Phase 9: Advanced Topics (Week 19-20)

9.1 Memory Management

- Garbage collection
- Reference counting
- Memory profiling
- Optimization techniques

YouTube Resources:

- **mCoding - Memory Management**

9.2 Concurrency Patterns

- ThreadPoolExecutor
- ProcessPoolExecutor
- Queue for thread communication
- Locks and synchronization

9.3 Networking

- Sockets programming basics
- Client-server architecture
- WebSockets

YouTube Resources:

- **Tech With Tim - Python Sockets**

🎯 Top Recommended YouTube Channels (Overall)

1. **Corey Schafer** - Best for clear, detailed explanations of Python fundamentals and intermediate topics
2. **ArjanCodes** - Excellent for software design, clean code, and advanced patterns
3. **mCoding** - Deep dives into Python internals and advanced concepts
4. **Tech With Tim** - Project-based learning, great for practical applications
5. **Real Python** - Professional-quality tutorials on various topics
6. **freeCodeCamp.org** - Full-length comprehensive courses
7. **Programming with Mosh** - Well-structured, professional tutorials

---

📚 Supplementary Resources

Documentation

- Official Python Documentation: https://docs.python.org/3/
- Python Enhancement Proposals (PEPs): https://peps.python.org/
- Real Python: https://realpython.com/

Practice Platforms

- LeetCode (for algorithms)
- HackerRank (Python challenges)
- Codewars (kata-style challenges)
- Project Euler (mathematical problems)

Books (Optional)

- "Fluent Python" by Luciano Ramalho
- "Effective Python" by Brett Slatkin
- "Python Cookbook" by David Beazley

🎓 Study Tips

1. **Code Daily**: Write Python code every single day, even if just 30 minutes
2. **Build Projects**: Apply concepts immediately in small projects
3. **Read Others' Code**: Explore open-source Python projects on GitHub
4. **Watch at 1.25-1.5x Speed**: Adjust based on your comprehension
5. **Take Notes**: Maintain a personal Python reference guide
6. **Practice Problems**: Solve 2-3 coding problems daily
7. **Review Regularly**: Revisit topics every 2-3 weeks

⏱️ Recommended Timeline

- **20 weeks total** for comprehensive coverage
- **2-3 hours daily** study time
- **1 project per week** minimum
- **Regular revision** of previous topics

Good luck with your Python developer journey! 🚀